Rotation Matrices — Everything in One Place

Contents

1 The mechanical recipe (read this first)

Everything below is one idea. A rotation matrix **tells you where the basis vectors go**, and you **put those images into the columns**. Concretely:

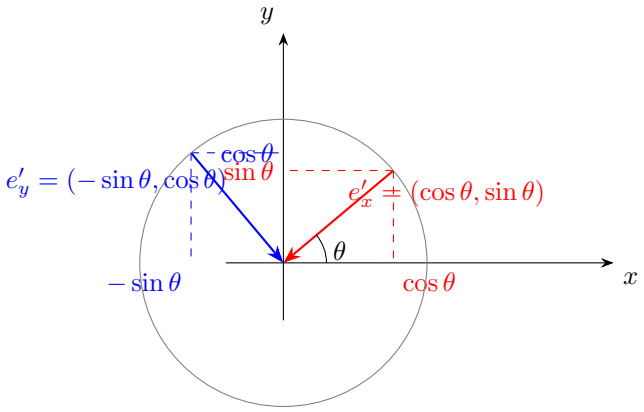
- The **columns** of R are the rotated basis vectors.
- You find each rotated basis vector by **orthogonal decomposition** (its projection onto the axes) — that is where the sin and cos come from.
- The **signs** come from the **direction** in which the basis vector moves.

Core idea: Rotate the basis vectors → decompose (project) them onto the axes → put the results in the columns of the matrix.

1.1 2D counterclockwise, read this way

The rotated e_x lands on θ above the x-axis; projecting gives $(\cos\theta, \sin\theta)$. The rotated e_y ends up "backwards-and-up"; projecting gives $(-\sin\theta, \cos\theta)$:

$$e_x \to \begin{pmatrix}\cos\theta\\ \sin\theta\end{pmatrix}, \qquad e_y \to \begin{pmatrix}-\sin\theta\\ \cos\theta\end{pmatrix} \qquad \Longrightarrow \qquad R_{2D}(\theta) = \begin{pmatrix}\cos\theta & -\sin\theta\\ \sin\theta & \cos\theta\end{pmatrix}.$$



The red arrow (image of e_x) is the **first column**; the blue arrow (image of e_y) is the **second column**. The blue one points "backward" (into $-x$), which is exactly why the minus sits in the top-right.

1.2 3D about $+y$, read this way

The same recipe. Positive rotation about $+y$ moves $+x$ toward $-z$, so the image of e_x has a **negative z-component** $(-\sin\theta)$; the image of e_z has a **positive x-component** $(+\sin\theta)$:

$$R_y(\theta) = \begin{pmatrix}\cos\theta & 0 & +\sin\theta\\ 0 & 1 & 0\\ -\sin\theta & 0 & \cos\theta\end{pmatrix}.$$

The direction of travel under the right-hand rule fixes the sign; the magnitude of the sideways component fixes the sin / cos scale.

1.3 Where this shortcut stops: an arbitrary axis $\hat{n}$

The "read the sign off the transition direction" shortcut works precisely **because** the axis is a coordinate axis — the other two basis vectors then lie flat *in* the rotation plane, so each one has a clean cos along its own direction and a clean $\pm\sin$ sideways. For a general rotation axis $\hat{n} = (n_1, n_2, n_3)$ no basis vector lies in the plane; each one splits into a **fixed part along** $\hat{n}$ and a **perpendicular part** that rotates, so the entries stop being plain $\pm\sin$ / cos and become products of the axis components:

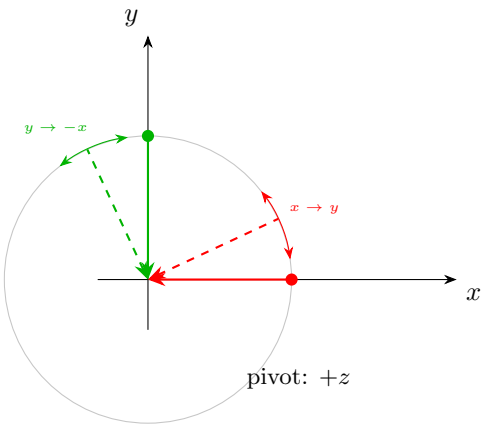
$$R_{\hat{n}}(\theta) = I + (\sin\theta)K + (1-\cos\theta)K^2, \qquad K = \begin{pmatrix}0 & -n_3 & n_2\\ n_3 & 0 & -n_1\\ -n_2 & n_1 & 0\end{pmatrix}.$$

This is **Rodrigues' rotation formula**. Same projection idea as before — you *compute* the components rather than read them off. Check: if $\hat{n}$ is a coordinate axis, one of n_1, n_2, n_3 is 1 and the others are 0, and the formula collapses exactly onto R_x, R_y or R_z. So this document is the special case of Rodrigues; the recipe "columns are the images of the basis vectors" remains true for the general case too.

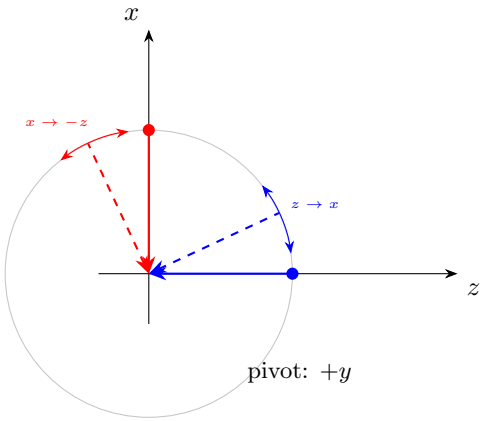
2 The three directions of basis movement (right-handed frame)

Every rotation matrix is the same story, repeated in a different plane. In the plane perpendicular to the pivot axis, positive rotation sweeps *counterclockwise* (viewed from the positive end of that axis), so each basis vector swings "forward" into the next axis of the cycle $x \to y \to z \to x$, and the axis it leaves behind picks up the minus. Below are the three pictures, drawn in each plane's own coordinate order (first axis horizontal, second vertical).

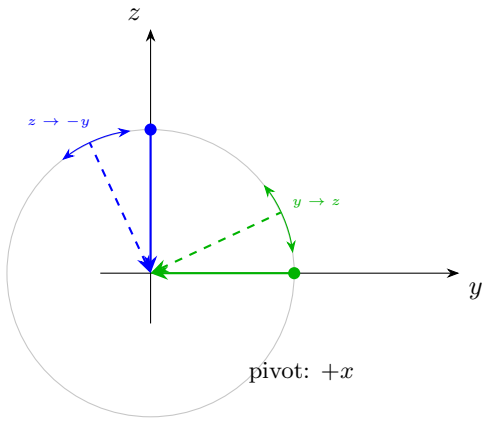
2.1 Rotation about $+z$ (the xy-plane): $x \to y$, $y \to -x$



2.2 Rotation about $+y$ (the zx-plane): $z \to x$, $x \to -z$



2.3 Rotation about $+x$ (the yz-plane): $y \to z$, $z \to -y$



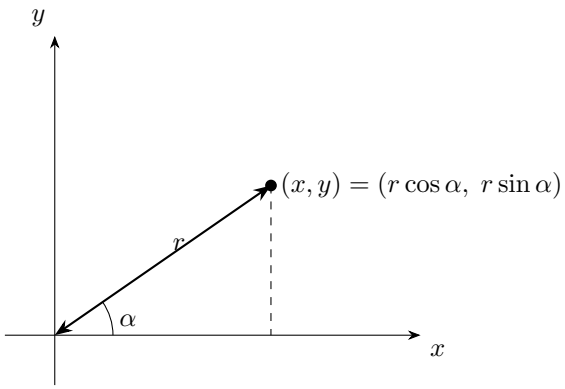
Printed this way, the pattern is unmistakable: the axis that swings *forward* into its right-handed successor carries $+\sin$; the axis that gets swung *backward* into the negative of its predecessor carries $-\sin$. Every matrix in this document is just those columns.

3 The 2D rotation (the engine)

3.1 Goal and polar setup

We want a matrix $R(\theta)$ so that multiplying (x, y) gives the point rotated counterclockwise by θ about the origin. Write (x, y) in polar form $x = r\cos\alpha$, $y = r\sin\alpha$ (radius r, angle α). Rotating by θ puts the point at angle $\alpha + \theta$ with the same r:

$$x' = r\cos(\alpha + \theta), \qquad y' = r\sin(\alpha + \theta).$$



3.2 Apply the angle-addition identities

Using $\cos(\alpha + \theta) = \cos\alpha\cos\theta - \sin\alpha\sin\theta$ and $\sin(\alpha + \theta) = \sin\alpha\cos\theta + \cos\alpha\sin\theta$,

$$x' = r\cos\alpha\cos\theta - r\sin\alpha\sin\theta = x\cos\theta - y\sin\theta,$$
$$y' = r\sin\alpha\cos\theta + r\cos\alpha\sin\theta = x\sin\theta + y\cos\theta,$$

using $x = r\cos\alpha$, $y = r\sin\alpha$. Collecting into a matrix:

$$\boxed{R_{2D}(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}}$$

3.3 Two worth-noting facts

- The columns are unit vectors (orthonormal): $R^{\mathsf{T}}R = I$, $\det R = 1$.
- Rotation preserves length and angle — why the L^2 norm and dot product are the "rotation-friendly" tools. Same θ as in $\|u\|\|v\|\cos\theta$.

4 2D rotation *is* rotation about the z-axis

A rotation in the xy-plane is exactly a rotation in $\mathbb{R}^3$ about the z-axis, with every point's z fixed. Embed the 2D matrix by appending the z-axis entries so z is untouched:

$$\boxed{R_z(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{pmatrix}}$$

Multiplying (x, y, z): the x, y part spins by the 2D block, and $z' = z$. The z-axis is real, not imaginary — it is the third direction perpendicular to the 2D picture ("out of the paper").

5 The same recipe: R_y and R_x

By identical reasoning, the pivot axis gets a $(0 \ldots 1 \ldots 0)$ row/column and the other two coordinates get the 2D block:

$$R_y(\phi) = \begin{pmatrix} \cos\phi & 0 & \sin\phi \\ 0 & 1 & 0 \\ -\sin\phi & 0 & \cos\phi \end{pmatrix}, \qquad R_x(\psi) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos\psi & -\sin\psi \\ 0 & \sin\psi & \cos\psi \end{pmatrix}.$$

6 The easy way to derive R_x and R_y

You reuse the 2D result in each perpendicular plane, just with different letters playing "x and y."

6.1 R_x = rotate the yz-plane (pivot x, untouched)

The plane's axes $\{y, z\}$ already read in coordinate order, so the 2D formula applies directly:

$$y' = y\cos\psi - z\sin\psi, \quad z' = y\sin\psi + z\cos\psi, \quad x' = x,$$

$$R_x(\psi) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos\psi & -\sin\psi \\ 0 & \sin\psi & \cos\psi \end{pmatrix}.$$

6.2 R_y = rotate the zx-plane (pivot y, untouched)

Here the plane's natural order is $\{z, x\}$; apply 2D to z (first) and x (second):

$$z' = z\cos\phi - x\sin\phi, \quad x' = z\sin\phi + x\cos\phi, \quad y' = y,$$

then rewrite in (x, y, z) order:

$$x' = x\cos\phi + z\sin\phi, \quad y' = y, \quad z' = -x\sin\phi + z\cos\phi,$$

$$R_y(\phi) = \begin{pmatrix} \cos\phi & 0 & +\sin\phi \\ 0 & 1 & 0 \\ -\sin\phi & 0 & \cos\phi \end{pmatrix}.$$

6.3 Why the sign flip in R_y is not arbitrary — the ordering story

The key rule: the 2D block $\begin{pmatrix}\cos & -\sin\\ \sin & \cos\end{pmatrix}$ applies correctly only to the pair that makes a *right-handed* frame with the pivot axis (i.e. the pair whose cross product points along positive pivot axis):

Rotation about	Plane axes	right-handed pair?
R_z	x, y	$x \times y = +z$ ✓
R_x	y, z	$y \times z = +x$ ✓
R_y	x, z	$x \times z = -y$ ✗, but $z \times x = +y$ ✓

The correct ordered pair for R_y is (z, x), not (x, z) — read "z is followed by x" *forward* (cyclic order $x \to y \to z \to x$). Applying the standard block to (z, x):

$$\begin{pmatrix}z'\\ x'\end{pmatrix} = \begin{pmatrix}\cos & -\sin\\ \sin & \cos\end{pmatrix}\begin{pmatrix}z\\ x\end{pmatrix},$$

which puts $-\sin$ in the (row z, col x) slot. Displaying that in the full (x, y, z) matrix: row z is the **bottom** row and col x is the **first** column, so $-\sin$ lands **bottom-left**.

Why it *looks* inverted (the "order swapped" resolution). If you wrote the block in (x, z) order (treating x as "first"), $x \times z = -y$ gives a *left-handed* pair and you'd wrongly get $-\sin$ top-right. The apparent "inversion" is purely: the plane's natural right-handed pair (z, x) does not match the display order (x, y, z), so the two off-diagonal entries swap places. Cropping R_y to its xz sub-block gives $\begin{pmatrix}\cos & \sin\\ -\sin & \cos\end{pmatrix}$ — the same block, just with x and z roles, hence rows/columns, exchanged vs. how R_z's block looks.

Cyclic relabeling: R_y *is* R_z in disguise. Cyclically rename $z \to x$, $x \to y$, $y \to z$ and "rotate the zx-plane about y" becomes "rotate the xy-plane about z":

$$R_y(\phi) = \begin{pmatrix}\cos & 0 & +\sin\\ 0 & 1 & 0\\ -\sin & 0 & \cos\end{pmatrix}_{(x,y,z)} \xrightarrow{(z,x,y)} \begin{pmatrix}\cos & -\sin & 0\\ \sin & \cos & 0\\ 0 & 0 & 1\end{pmatrix}_{(z,x,y)},$$

which is R_z-shaped. So nothing is special about R_y — it only *seems* inverted because the display order (x, y, z) is out of step with the rotation's natural pair (z, x).

Never-perform reliability check at $\phi = 90°$. A 90° rotation about y sends $z \to +x$ and $x \to -z$. So column 3 (image of z) must be $(1, 0, 0)$, and column 1 (image of x) must be $(0, 0, -1)$: $R_y(90°) = \begin{pmatrix}0 & 0 & 1\\ 0 & 1 & 0\\ -1 & 0 & 0\end{pmatrix}$, which reproduces the matrix exactly. This physical axis check recovers R_y without pattern-matching a sign table.

7 Roll, pitch and yaw (the human frame)

Body frame: x = forward (nose), y = out the right, z = down.

Name	Axis	Matrix	Rotates the plane
Yaw	z	$R_z(\theta)$	xy
Pitch	y	$R_y(\phi)$	zx
Roll	x	$R_x(\psi)$	yz

8 Order matters: Euler angles and the ZYX convention

8.1 There is no single universal order

The order is a convention, and 12 intrinsic orders exist. Two families:

- **Proper Euler angles**: same axis for 1st and 3rd rotation (e.g. ZXZ, ZYZ) — classical mechanics / quantum spin.
- **Tait–Bryan angles**: three different axes (e.g. ZYX, XYZ) — aerospace / navigation / robotics. Roll–pitch–yaw is Tait–Bryan.

Euler angles are also *non-unique* and subject to **gimbal lock** (a lost degree of freedom when the middle angle aligns two axes) — why robotics often uses quaternions.

8.2 ZYX (intrinsic) order = yaw, then pitch, then roll

Each subsequent rotation happens about the axis that has just been rotated (body frame):

1. **Yaw**: about the body's z-axis, by θ.
2. **Pitch**: about the body's *new* y-axis, by ϕ.
3. **Roll**: about the body's *new* x-axis, by ψ.

The total rotation is the right-to-left product

$$\boxed{R = R_x(\psi)\, R_y(\phi)\, R_z(\theta)}$$

applied as $v' = R\, v$.

8.3 Why the matrix closest to the vector acts first

Matrix–vector multiplication groups right-to-left: $R\, v = R_x\big(R_y\,(R_z\, v)\big)$, so $R_z\, v$ is computed first, then R_y, then R_x — matching intrinsic ZYX. Since v sits on the right of $R_x R_y R_z$, the **first-applied matrix goes on the right**:

> "ZYX" describes *application* order (Z first); the matrix string mirrors it by putting the first-applied factor rightmost. Write left-to-right, read right-to-left. Swapping to $R_z R_y R_x$ yields the reversed (extrinsic) reading.

8.4 Fixed vs rotating axes

Rotating about the *moving* axes (intrinsic ZYX) gives the *same* final geometry as rotating about the *fixed* axes in reversed order:

$$\text{intrinsic } z, y, x \quad \Longleftrightarrow \quad \text{extrinsic } x, y, z.$$

9 Complete table and summary

Rotation	Matrix	Plane
$R_z(\theta)$	$\begin{pmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{pmatrix}$	xy
$R_y(\phi)$	$\begin{pmatrix} \cos\phi & 0 & \sin\phi \\ 0 & 1 & 0 \\ -\sin\phi & 0 & \cos\phi \end{pmatrix}$	zx
$R_x(\psi)$	$\begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos\psi & -\sin\psi \\ 0 & \sin\psi & \cos\psi \end{pmatrix}$	yz

- 3D rotation about any axis = the 2D rotation in the perpendicular plane, with that axis left untouched.
- R_z is literally the 2D matrix plus a $0, 0, 1$ row/column.
- The sign in each matrix lives at "first-row/second-column of the block" — it follows the plane's axes, never an arbitrary choice.
- General 3D orientation = an *ordered* product; ZYX means z (yaw), then y (pitch), then x (roll), about the already-rotated axes.